# MINDSTRIX – TASK 1: GOOGLE EARTH ENGINE AND SATELLITE DATA

1. **Think of Google Earth Engine like this:**

- A giant **Google Drive of Earth data**
- A **supercomputer**
- A **map like Google Maps**

All combined into one tool.

2. **What Problem Does It Solve?**
   Normally, if you want satellite data:

   Files are **huge (GBs–TBs)**
   Hard to process
   Need powerful computers
   Google Earth Engine removes all that.
   You don't download data - you just **use it online instantly.**
   **Basically everything is cloud based it runs on Google's GPUs.**

3. **What Can You Actually DO With It?**

Let's keep it real and practical. **See the Earth (Like Google Maps, but smarter)**

Zoom anywhere in the world

See satellite images

Compare **before vs after**

Example: How Pune looked 10 years ago vs now

4. **Google Earth Engine – Features (Simple & Short)**

- Huge collection of satellite and earth data (Landsat, Sentinel, weather, etc.)
- Works online (no need to download data)
- Cloud processing (runs on Google servers, no powerful laptop needed)
- Interactive maps for easy visualization
- Time analysis (see changes over years)
- Supports JavaScript and Python coding
- Easy data layering (combine multiple datasets on map)
- Built-in tools for analysis (like crop health, water detection)
- Machine learning support for predictions
- Fast processing of very large datasets
- Free for students and research use
- Can export results as images, maps, or data

5. **LANDSAT VS SENTINEL**

| Feature | Landsat | Sentinel |
|---|---|---|
| Developed By | NASA + USGS | European Space Agency (ESA) |
| Program Start | 1972 | ~2015 (Copernicus Programme) |
| Purpose | Long-term Earth monitoring | High-resolution, real-time monitoring |
| Resolution | 30 meters | 10 meters (Sentinel-2) |
| Image Clarity | Moderate (less detailed) | High (more detailed) |
| Revisit Time | 16 days | 5 days (Sentinel-2) |
| Data Availability | 50+ years of data | Recent data (~10 years) |
| Spectral Bands | Fewer bands | More bands (13 in Sentinel-2) |
| Cloud Handling | Cannot see through clouds | Sentinel-1 radar works through clouds & night |
| Types | Mainly optical satellites | Multiple types (Sentinel-1, 2, 3, 5P) |
| Best For | Historical analysis, climate studies | Agriculture, disaster monitoring, real-time analysis |

| Feature | Landsat | Sentinel |
|---|---|---|
| Agriculture Use | Basic crop monitoring | Advanced crop health (NDVI, moisture, etc.) |
| Weather Conditions | Affected by clouds | Works in all weather (with radar) |
| Data Frequency | Slower updates | Faster updates |
| Use in GEE | Used for long-term trends | Widely used for modern applications |

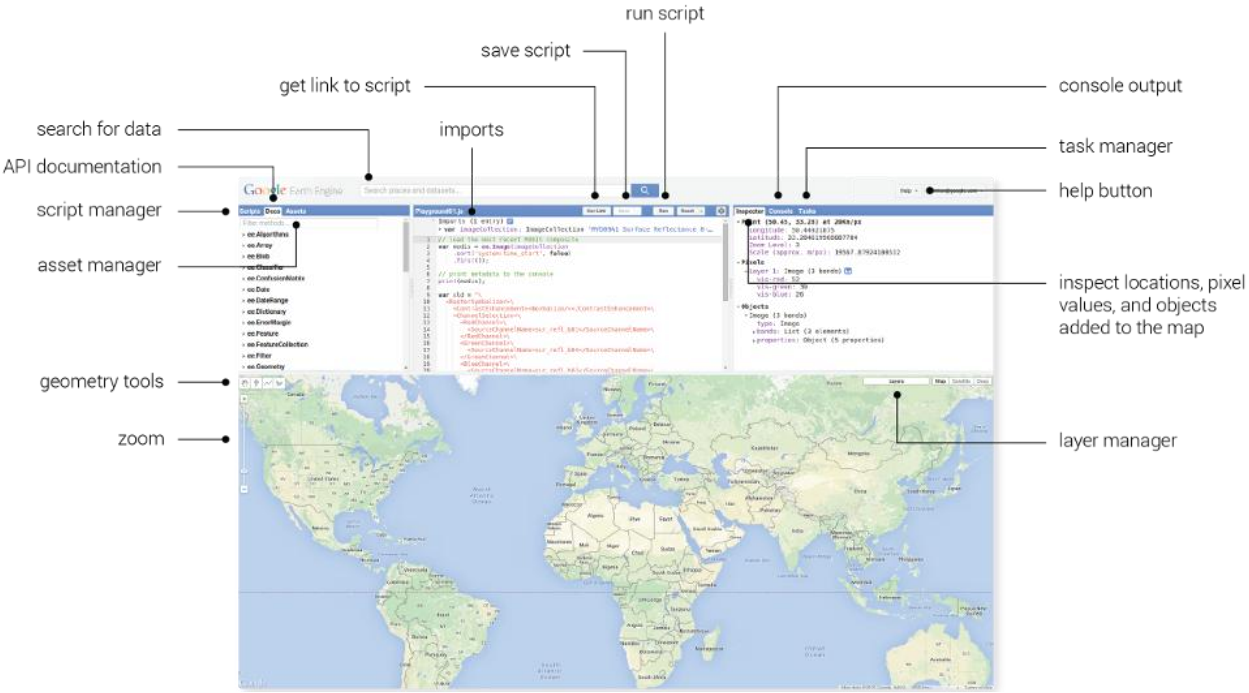
**OPTICAL SATELLITES**

1. Sun gives light
2. Light hits Earth (land, water, plants)
3. Earth reflects that light
4. Satellite captures it
5. Image is formed (like a normal photo)

**RADAR SATELLITES**

1. Satellite sends its own signal (radio waves)
2. Signal hits Earth
3. Signal bounces back
4. Satellite receives it
5. Image/data is created

## 6. GOOGLE EARTH ENGINE OVERVIEW




**Google Earth Engine Interface (Based on Your Image)**

**1. Search Bar (Top Center)**

- Used to search:
  - Datasets (Sentinel, Landsat, etc.)
  - Functions
- Example: type Sentinel to find datasets

---

**2. Scripts Panel (Top Left)**

- Shows your saved scripts
- You can:
  - Create new scripts
  - Open old projects
- Acts like your "project files"

---

**3. Code Editor (Center Top)**

- Main area where you write code
- You define:
  - Dataset
  - Area
  - Analysis (NDVI, etc.)
- This is where all logic is written

---

**4. Run Button (Top Above Code)**

- Executes your code
- After clicking:
  - Results appear on map
  - Output shows in console

---

**5. Docs / Help (Top Right Area)**

- Provides documentation
- Helps you understand functions and syntax

---

**6. Console Panel (Top Right)**

- Shows:
  - Output values
  - Errors
  - Logs
- Used to check if code is working

---

**7. Inspector Panel (Right Side)**

- Click on map → shows pixel data
- Example:
  - NDVI value
  - Band values
- Useful for checking results

---

**8. Map Panel (Bottom Large Area)**

- Main visualization area
- Displays:
  - Satellite images
  - Analysis results
- You can zoom, pan, and explore locations

---

**9. Layers (On Map)**

- Shows added layers (datasets/results)
- You can:
  - Turn layers on/off
  - Adjust visibility

---

**10. Geometry Tools (On Map – Drawing Tools)**

- Used to draw:
  - Points
  - Polygons

    - Regions
- Helps select area of interest

---

**Simple Flow Using These Sections**

1. Search dataset (Search bar)
2. Write code (Code editor)
3. Run code (Run button)
4. See result (Map panel)
5. Check output (Console)
6. Inspect values (Inspector)

---

**One-Line Understanding**

- Left = project + scripts
- Center = coding
- Right = output + inspection
- Bottom = visualization

7. **Writing First Script**

```
var image = ee.ImageCollection('COPERNICUS/S2_SR')
  .filterDate('2023-01-01', '2023-01-31')
  .filterBounds(ee.Geometry.Point([73.85, 18.52]))
  .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 20))
  .median();

Map.centerObject(ee.Geometry.Point([73.85, 18.52]), 10);

Map.addLayer(image, {
  min: 0,
  max: 3000,
  bands: ['B4', 'B3', 'B2']
}, 'Correct Image');
```

**- Load the dataset**

var image = ee.ImageCollection('COPERNICUS/S2_SR')

- ee.ImageCollection → a collection (group) of many satellite images
- 'COPERNICUS/S2_SR' → Sentinel-2 dataset (surface reflectance)

You are telling GEE:

"Give me Sentinel-2 satellite images"

---

**- Filter by date**

.filterDate('2023-01-01', '2023-01-31')

- Selects images only from **January 2023**

Meaning:

"I only want images taken in this time period"

---

**- Filter by location**

.filterBounds(ee.Geometry.Point([73.85, 18.52]))

- Filters images that cover this location
- Coordinates = Pune

Meaning:

"Give me images that include this location"

---

**- Filter clouds**

.filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 20))

- Keeps only images with **less than 20% clouds**

Meaning:

"Remove cloudy images"

---

**- Combine images**

.median();

- Takes all filtered images
- Combines them into **one image**

How:

- For each pixel → takes the **median value**

Meaning:

"Create one clean image from multiple images"

---

**- Center the map**

Map.centerObject(ee.Geometry.Point([73.85, 18.52]), 10);

- Moves map to Pune
- 10 = zoom level

Meaning:

"Show this location on the map"

---

**- Display the image**

```
Map.addLayer(image, {
  min: 0,
  max: 3000,
```

bands: ['B4', 'B3', 'B2']
}, 'Correct Image');

**Breakdown:**

**image**

- The processed satellite image

---

**min: 0, max: 3000**

- Controls brightness/contrast

Meaning:

"Adjust how the image looks"

---

**bands: ['B4', 'B3', 'B2']**

- Chooses color channels:
  - B4 → Red
  - B3 → Green
  - B2 → Blue

Meaning:

"Show image in natural colors (like real world)"

---

**'Correct Image'**

- Name of the layer on map

## 8. DATASETS

| Dataset | Type | Used For |
|---|---|---|
| Sentinel-2 (COPERNICUS/S2_SR) | Optical satellite | Crop health, vegetation, land use |
| Sentinel-1 (COPERNICUS/S1_GRD) | Radar satellite | Soil moisture, flood detection, irrigation |
| Landsat 8/9 (LANDSAT/LC08/C02/T1_L2) | Optical satellite | Long-term change, urban growth, climate study |
| MODIS (MODIS/006/MOD13Q1) | Satellite (low res, frequent) | Large-scale vegetation, NDVI trends |

| Dataset | Type | Used For |
|---|---|---|
| CHIRPS (UCSB-CHG/CHIRPS/DAILY) | Rainfall data | Rain analysis, drought monitoring |
| ERA5 (ECMWF/ERA5_LAND/HOURLY) | Climate data | Temperature, wind, weather patterns |
| JRC Water (JRC/GSW1_4/GlobalSurfaceWater) | Water dataset | Rivers, lakes, water change over time |
| ESA WorldCover (ESA/WorldCover/v100) | Land cover map | Forest, urban, agriculture classification |
| SRTM (USGS/SRTMGL1_003) | Elevation data | Height, slope, terrain analysis |
| Sentinel-5P | Atmospheric satellite | Air pollution ($NO_2$, CO, ozone) |

9. **GEE API's**

Google Earth Engine APIs (Detailed Explanation)
What is an API in GEE?
API means a way to interact with Google Earth Engine using code instead of clicking buttons.

---

**1. JavaScript API**
Where it is used

- Inside the GEE Code Editor (browser)

What it does

- Load datasets
- Filter data (date, location, clouds)
- Process images
- Display results on map

Key characteristics

- Runs directly in browser

- No installation required
- Best for beginners and learning
- Immediate visualization

**Example**

```
var image = ee.ImageCollection('COPERNICUS/S2_SR')
  .filterDate('2023-01-01', '2023-01-31')
  .median();

Map.addLayer(image);
```

When to use

- Learning GEE
- Prototyping ideas
- Hackathons
- Visualization tasks

---

**2. Python API**

Where it is used

- Python environment
- Jupyter Notebook
- Google Colab

What it does

- Same operations as JavaScript API
- Allows integration with:
    - Machine learning
    - Backend systems
    - Data pipelines

Key characteristics

- Requires setup (pip install earthengine-api)
- Works well with libraries like NumPy, Pandas
- Suitable for automation

**Example**

```
import ee
ee.Initialize()

image = ee.ImageCollection('COPERNICUS/S2_SR') \
    .filterDate('2023-01-01', '2023-01-31') \
    .median()
```

When to use

- AI/ML integration
- Backend development
- Automated workflows

---

**3. REST API**

What it is

- A web-based API using HTTP requests

What it does

- Allows external applications to communicate with GEE

Key characteristics

- Used in production systems
- Not beginner-friendly
- Requires authentication and request handling

When to use

- Web applications
- Mobile apps
- Backend services

---

Core Functional APIs inside GEE

These are the main building blocks you use in both JavaScript and Python APIs.

---

**1. Image API**

- Works with a single image

ee.Image()

Use:

- Processing one satellite image

---

**2. ImageCollection API**

- Works with multiple images

ee.ImageCollection()

Use:

- Time-series analysis
- Combining images

---

**3. Geometry API**

- Defines location or region

ee.Geometry.Point()

ee.Geometry.Polygon()

Use:

- Selecting area of interest

---

**4. Filter API**

- Filters data

ee.Filter.date()
ee.Filter.lt()
Use:

- Date filtering
- Cloud filtering

---

**5. Reducer API**

- Aggregates data

ee.Reducer.mean()
ee.Reducer.median()
Use:

- Creating single image from many
- Statistical analysis

---

**6. Export API**

- Exports results

Export.image.toDrive()
Use:

- Save images
- Download processed data

---

**Summary Table**

| API Type | Purpose | Where Used |
|---|---|---|
| JavaScript API | Learning, visualization | GEE Code Editor |
| Python API | Automation, AI integration | Colab, Jupyter |
| REST API | App integration | Backend/web apps |

## 10. IMPORTANT TERMS

| Term | Full Form | Meaning |
|---|---|---|
| NDVI | Normalized Difference Vegetation Index | Crop health |
| NDWI | Normalized Difference Water Index | Water content |
| EVI | Enhanced Vegetation Index | Advanced vegetation |
| NDMI | Normalized Difference Moisture Index | Plant moisture |
| NIR | Near Infrared | Plant health signal |
| SAR | Synthetic Aperture Radar | Radar imaging |
| SWIR | Shortwave Infrared | Moisture/stress |
| RGB | Red Green Blue | Normal image |

**CROP IDENTIFICATION PARAMETER**

1. What YOU are building (clear goal)
A system that takes satellite data and tells:
"This field has wheat / rice / sugarcane."

---

**2. Where your data will come from**
**Primary source (you will use this)**
Google Earth Engine datasets
Main dataset:

- COPERNICUS/S2_SR → Sentinel-2 (optical)

Supporting dataset:

- COPERNICUS/S1_GRD → Sentinel-1 (radar)

---

What this means (simple)

- You are not collecting data manually
- GEE already stores satellite data
- You just query it using code

---

**3. Foundation: Bands (re-explained properly)**
A band = one type of light
Important Sentinel-2 bands:

| Band | Full Name | Use |
|---|---|---|
| B2 | Blue | Water, atmosphere |
| B3 | Green | Vegetation |
| B4 | Red | Plant absorption |
| B8 | NIR (Near Infrared) | Plant health (critical) |
| B11 | SWIR (Shortwave Infrared) | Moisture |
| B12 | SWIR | Soil stress |

---

Key understanding
Healthy plants reflect NIR strongly
Unhealthy plants do not
This is the base of everything.

---

**4. Parameters (core of your task)**
These are derived using bands.

---

**4.1 NDVI**
Normalized Difference Vegetation Index
Formula:
(NIR - RED) / (NIR + RED)
Using bands:

- NIR → B8
- RED → B4

---

Meaning:

- High NDVI → healthy crops
- Low NDVI → dry or no crops

---

**4.2 NDWI**
Normalized Difference Water Index
Formula:
(GREEN - NIR) / (GREEN + NIR)
Using:

- B3 and B8

---

Meaning:

- High → water present
- Useful for rice (paddy)

---

**4.3 NDMI**
Normalized Difference Moisture Index
Meaning:

- Measures moisture in vegetation

---

**4.4 EVI**
Enhanced Vegetation Index
Meaning:

- More accurate than NDVI
- Handles dense crops better

---

**4.5 SAR Features (Sentinel-1)**
Full form:
Synthetic Aperture Radar
Values:

- VV
- VH

---

Meaning:

- Soil moisture
- Surface structure

---

**5. Most Important Concept: Time-Series**

You will NOT use a single image

You will track values over time.

---

**Example:**

| Month | NDVI |
|---|---|
| June | 0.2 |
| July | 0.5 |
| August | 0.8 |

---

Why this matters

Each crop behaves differently:

| Crop | Pattern |
|---|---|
| Rice | Water + sharp rise |
| Wheat | Smooth growth |
| Sugarcane | Long steady growth |

---

**6. Complete Non-Technical Workflow**

1. Select area (farm/region)
2. Get satellite images (multiple dates)
3. Calculate NDVI, NDWI, etc.
4. Track values over time
5. Observe pattern
6. Identify crop

---

**7. Technical Workflow (Actual System)**

Step 1: Data collection

- Load Sentinel-2 data

- Filter by date and location

---

**Step 2: Preprocessing**

- Remove clouds
- Combine images

---

**Step 3: Feature generation**

For each image:

- NDVI
- NDWI
- NDMI

---

**Step 4: Time-series creation**

For each farm:

| Feature | Value |
|---|---|
| NDVI_June | 0.2 |
| NDVI_July | 0.5 |
| NDVI_Aug | 0.8 |

---

**Step 5: Add radar features**

VV:

- Vertical transmit → Vertical receive

VH:

- Vertical transmit → Horizontal receive

Simple meaning:

- Different ways radar signal reflects
- Helps detect:
    - Soil moisture
    - Surface structure

---

**Step 6: Create dataset**

Each row = one farm

---

**Step 7: Train model**

Model input:

- NDVI time-series
- NDWI

- Radar data

  Output:
- Crop type

---

Step 8: Prediction

New farm → model predicts crop

---

**8. What YOU specifically will do**

Your responsibility (crop identification):

- Compute NDVI, NDWI, NDMI
- Build time-series
- Prepare features
- Feed to ML model

---

**9. Full Pipeline (Final View)**

GEE (Sentinel Data)

↓

Band Processing

↓

Index Calculation (NDVI, NDWI, NDMI)

↓

Time-Series Creation

↓

Feature Table

↓

ML Model (Random Forest)

↓

Crop Prediction

---

**10. Key Insight (Very Important)**

Single image ≠ enough

Time pattern = identity of crop

---

**11. What makes your system strong**

- Multi-source data (optical + radar)
- Time-series instead of static image
- Feature engineering (indices)

---

**12. Final one-line understanding**

You are converting satellite light signals into time-based patterns to classify crops using AI

**WHERE WILL WE GET THIS BEHAVIOUR DATA?**

1. What is this "crop behavior data"?
It is knowledge like:

- Rice needs standing water
- Wheat grows in winter
- Sugarcane grows slowly over months

This is called:
Crop Phenology
(Phenology = study of plant growth stages over time)

---

**2. Why This is Important for YOU**
Satellite only gives:

- NDVI values
- Moisture
- Images

But YOU must know:
"What pattern belongs to which crop"
That mapping = your intelligence layer

---

**3. Crop Patterns (Very Important Table)**
**Major Indian Crops (Simplified)**

| Crop | Season | Growth Speed | Water Need | NDVI Pattern | Special Signal |
|---|---|---|---|---|---|
| Rice (Paddy) | Kharif (Jun–Oct) | Fast | Very High | Sharp increase | High NDWI (water present) |
| Wheat | Rabi (Nov–Apr) | Medium | Medium | Smooth curve | Moderate NDVI |
| Sugarcane | Year-round | Slow | High | Long stable | Long duration crop |

| Crop | Season | Growth Speed | Water Need | NDVI Pattern | Special Signal |
|---|---|---|---|---|---|
| | | | | high NDVI | |
| Maize | Kharif/ Rabi | Fast | Medium | Quick rise then drop | Short cycle |
| Cotton | Kharif | Medium | Medium | Gradual increase | Long mid-stage |
| Soybean | Kharif | Fast | Medium | Short sharp peak | Short lifecycle |

---

**4. Time-Series Patterns (Core Idea)**

Rice (Paddy)

- Starts low
- Water present → NDWI high
- NDVI increases quickly

Signature:

- Water + fast growth

---

Wheat

- Slow start
- Gradual increase
- Smooth curve

Signature:

- No water flooding
- Stable growth

---

Sugarcane

- Very long growth
- NDVI stays high for months

Signature:

- Long duration + constant high NDVI

---

Maize

- Quick growth
- Quick harvest

Signature:

- Short NDVI peak

---

5. Where Do You Get This Data?

This is your main question.

---

**1. Government Sources (India)**

ICAR (Indian Council of Agricultural Research)

- Crop calendars
- Growth stages

---

IMD (Indian Meteorological Department)

- Rainfall + season data

---

State agriculture portals

- Crop schedules

---

**2. Research Papers**

Search:

- “NDVI time series crop classification India”
- “crop phenology remote sensing”

---

**3. FAO (Food and Agriculture Organization)**

- Global crop calendars
- Growth duration

---

**4. Kaggle / Datasets**

Search:

- crop classification dataset
- satellite crop dataset

---

Important Reality

No one gives you “perfect ready dataset”

You must:

- Combine knowledge
- Combine satellite data

- Build your own dataset

---

**6. How YOU Will Use This**

**Step 1: Define crop rules (logic layer)**

Example:

Rice:

- High NDWI
- NDVI rises quickly
- Occurs in Kharif

---

Wheat:

- No standing water
- Smooth NDVI curve
- Rabi season

---

**Step 2: Match with satellite data**

From GEE you get:

- NDVI over time
- NDWI
- Moisture

---

**Step 3: Build Feature Pattern**

Example:

Farm A:

NDVI pattern → fast rise

NDWI → high

Season → Kharif

→ Crop = Rice

---

**7. Two Approaches You Can Use**

---

🔹 Approach 1: Rule-Based (Beginner)

You define rules:

- If NDWI high → water crop
- If NDVI long stable → sugarcane

---

🔹 Approach 2: Machine Learning (Better)

- Feed patterns into model
- Model learns differences

---

**8. Your Final Pipeline (Correct Version)**

Crop Knowledge (Phenology)

+

Satellite Data (GEE)

↓

Feature Extraction (NDVI, NDWI, Time-Series)

↓

Pattern Matching / ML Model

↓

Crop Identification

---

**9. Most Important Insight**

Satellite gives signals

Crop knowledge gives meaning

---

**10. What Makes Your Project Strong**

Most people:

- Only use NDVI

YOU will:

- Use NDVI + NDWI + time-series
- Combine with crop behavior knowledge

---

**Final One-Line Understanding**

Crop identification = matching satellite patterns with real crop growth behavior